

Pac-Man : IA de poursuite

Génération de labyrinthes, variabilité des IA et mesure de performances

Ikram Benchalal Nada Zina Aya Haddoun

Université Côte d'Azur

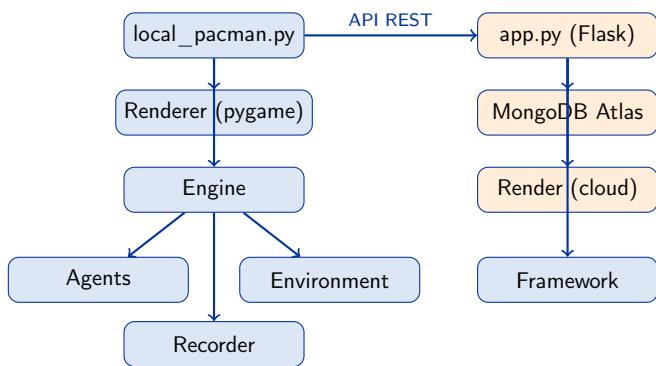
Superviseur : Gilles Menez

2025 – 2026



- 1 Architecture
- 2 Base de données et API
- 3 Modélisation du jeu
- 4 IA de poursuite
- 5 Mode solo et replay
- 6 Tests et résultats
- 7 Conclusion

Architecture du projet



Client local

- Interface graphique pygame
- Moteur de jeu (score, vies, collisions)
- 5 algorithmes de poursuite
- Enregistrement JSON

Serveur distant

- Génération de labyrinthes
- Stockage et notation MongoDB
- Benchmark des IA

MongoDB Atlas

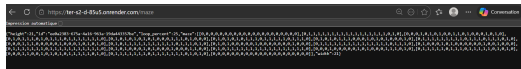
- Collection mazes dans pacman_db
- Grille, dimensions, note moyenne, votes

Cache local SQLite

- Sauvegarde hors-ligne

Déploiement

- Render + CI/CD GitHub Actions



Routes API

Route

Rôle

GET /maze

Générer un labyrinthe

POST /maze/id/rate

Noter (0 à 5)

GET /ai/algorithms

Liste des IA

POST /ai/benchmark

Comparer les IA

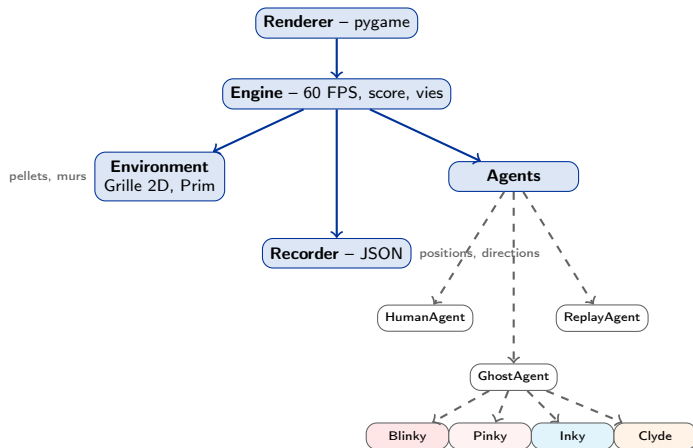


Notation en jeu



Note dans MongoDB

Modélisation du jeu



Pac-Man, fantômes, pellets et score en temps réel

Algo	Caractéristique
BFS	Plus court chemin garanti
A*	Heuristique Manhattan, optimal
DFS	Non-optimal, erratique
UCS	Coût uniforme (= BFS sur grille)
MCTS	Simulations Monte Carlo

Options avancées

Prédiction

- Anticipe la position future de Pac-Man
- Paramètre k (nombre de pas)

Coopération

- Fantômes évitent le même chemin
- Meilleure couverture du labyrinthe

MCTS

- 30 itérations, profondeur 15
- Approche non-déterministe

Fantôme	Couleur	Rôle	Ciblage
Blinky	Rouge	Chasseur	Cible Pac-Man
Pinky	Rose	Embuscade	4 cases devant
Inky	Cyan	Flanc	Tenaille avec Blinky
Clyde	Orange	Aléatoire	Fuit si < 8 cases

Les 4 fantômes avec leurs couleurs distinctes

Cycle des modes

- 1 Caged (10s) → Scatter (7s) → Chase (20s)
- 2 Alternance scatter/chase

Frightened : power pellet → combo 200-1600 pts

Mode solo et replay

Solo (touche 1)

- Sans fantômes
- Exploration libre
- Trajectoire enregistrée

Replay (touche 2)

- Rejoue le trajet
- 4 fantômes IA
- Changer d'algo avec 2

Observation

- IA affichée à l'écran
- Tick de capture noté
- Compare les IA



Mode solo



Mode replay

Tests et résultats

51 tests automatisés

Module	Tests
Générateur	10
API Flask	5
Jeu	24
Framework	12
Total	51

```
C:\Users\nadaz\OneDrive\Bureau\TER_S2_Doc\Users\nadaz\OneDrive\Bureau\TER_S2_Doc\Scripts\python.exe -m pyte
st -g
555 ..... [100%]
52 passed, 3 skipped in 34.95s

C:\Users\nadaz\OneDrive\Bureau\TER_S2_Doc\Users\nadaz\OneDrive\Bureau\TER_S2_Doc\Scripts\python.exe start.p
y --seeds 1 --steps 10
pygame 2.6.1 (SDL 2.28.4, Python 3.10.11)
Hello from the pygame community. https://www.pygame.org/contribute.html
== test des methodes sur plusieurs seeds ==

-- Summary --
bfs_reactive: capture_rate=66.7% avg_capture_tick=9.00
astar_reactive: capture_rate=66.7% avg_capture_tick=9.00
ucs_reactive: capture_rate=66.7% avg_capture_tick=9.00
bfs_predictive_k3: capture_rate=66.7% avg_capture_tick=9.00
astar_predictive_k3: capture_rate=66.7% avg_capture_tick=9.00
team_bfs_coop: capture_rate=66.7% avg_capture_tick=9.00
team_astar_coop_k3: capture_rate=66.7% avg_capture_tick=9.00
team_astar_roles: capture_rate=66.7% avg_capture_tick=9.00
dfs_reactive: capture_rate=66.7% avg_capture_tick=9.00
dfs_reactive: capture_rate=66.7% avg_capture_tick=9.00
('runs': [{'results': {'astar_predictive_k3': {'capture_tick': 9,
'captured': True,
'lives_left': 0,
'score': 0,
'ticks_played': 41},
'astar_reactive': {'capture_tick': 9,
'captured': True,
'lives_left': 0,
'score': 0,
'ticks_played': 41}}}]
```

Benchmark : ticks avant capture

Stratégie	Ticks	Capture
BFS/A*/UCS réactif	37.7	100%
BFS prédictif (k=3)	37.0	100%
A* prédictif (k=3)	17.0	100%
BFS coopératif	13.3	100%
A* coopératif (k=3)	14.3	100%
MCTS réactif	9.7	100%
A* rôles	13.3	100%
DFS réactif	229.5	67%

Fonctionnalités livrées

- ➊ Générer un labyrinthe depuis le cloud
- ➋ Noter les labyrinthes (touches 0-7)
- ➌ Représentation graphique complète
- ➍ Jouer sans fantômes (enregistrement)
- ➎ Rejouer avec fantômes (choix IA)
- ➏ Enregistrement avec fantômes
- ➐ Ticks avant capture affichés

Ce qu'on retient

- 5 algorithmes implémentés et comparés
- BFS et A* les plus efficaces
- DFS : limites du non-optimal
- MCTS : approche non-déterministe
- Architecture modulaire et testable

Technologies

Python, Flask, pygame, MongoDB Atlas, Render, GitHub Actions, pytest